水箱液位神经网络控制系统设计报告

本次作业中我尝试了两种控制方案，首先我尝试了监督学习模仿的控制策略，但发现效果相比训练时的参考GT控制(第一次作业时的模糊控制作为监督信号))的效果差那么一点点，我就从 强化学习自主优化这个想法出发，设计了一套基于强化学习的控制策略，最终效果不仅超过了前者，也超过了第一次的模糊控制策略。下面我将详细介绍我本次作业整个探索过程中的设计思路、遇到的瓶颈以及最终的解决方案。

一、问题描述与系统建模

1.1 控制对象

本任务的被控对象是一个单入单出的水箱系统。根据环境配置文件我们可以知道水箱的液位动力学方程为：

```
dH/dt = ip · u - op · sqrt(H)
```

其中：

- `H` 为当前水位（单位：cm）
- `u` 为阀门控制量（范围 `[0, 15]`）
- `ip = 2.5` 为进水管系数
- `op = 2.5` 为出水管系数

出水速率与水位的平方根成正比，这是一个典型的**非线性系统**。

1.2 控制目标

仿真总时长为 15 秒，采样步长 `dt = 0.01s`（共 1500 步）。控制目标分两阶段：

阶段	时间范围	目标水位
Phase 1	0 ~ 7.5s	`sp = 25`
Phase 2	7.5 ~ 15s	`sp = 10`

初始水位约为 5~6 cm（`rand()+5`）。

1.3 评分机制

系统的评分函数从初始分 10000 分开始，每一步扣分：

```
Score_{t+1} = Score_t - |H - sp|^2 · dt - |u_eq - u|^2 · dt
```

其中 `u_eq = op · sqrt(sp) / ip` 是目标水位下的平衡阀门开度。评分越高代表跟踪误差越小、控制量越接近平衡点。

二、阶段一：模仿学习（Imitation Learning）

2.1 设计思路

我最初的想法是：既然第一次作业已经设计好了一个已有的模糊控制器，不如直接让神经网络去**模仿学习** 。这就是经典的监督学习路线——把模糊控制器当作"老师"，让 NN 去学习老师的输入-输出映射。

2.2 神经网络结构设计

我设计了一个 4 层全连接网络，并且完全**手写了前向推理引擎**，没有依赖任何深度学习工具箱：

```
输入层 (4) → 隐藏层1 (64) → 隐藏层2 (32) → 隐藏层3 (16) → 输出层 (1)
```

激活函数选用 **Leaky ReLU**（`alpha = 0.01`），总参数量为 2945。

输入特征工程是我花了较多心思设计的部分。我没有简单地把原始水位丢进去，而是构造了 4 个物理含义明确的特征：

```
% 构造 NN 输入特征
e    = sp - H;                          % 跟踪误差
de   = e(t) - e(t-1);                   % 误差变化率（离散微分）
sp   = targetHeight;                    % 目标水位
u_eq = op * sqrt(max(sp, 0)) / ip;      % 伯努利平衡点前馈参考
```

其中 `u_eq` 这一项是我根据稳态条件 `dH/dt = 0` 推导出来的前馈参考值。当系统处于平衡态时，`u_eq = op · sqrt(sp) / ip`。将它直接作为 NN 的输入特征，相当于把物理先验知识注入到了网络中，大幅降低了 NN 的学习难度——网络只需要学习在前馈基础上做微调即可。

2.3 训练数据生成

数据生成是模仿学习的核心，我做了两个关键改进：

改进一：多工况覆盖。我设计了 50 个 episode，前 10 个用固定的极端初始水位（0.5, 1, 2, 3, 5, 15, 20, 28, 30, 35），后 40 个用 `rand()*35` 随机采样。这确保 NN 在各种初始条件下都见过"老师"怎么做：

```
% 覆盖极端初始水位 + 随机值
fixed_inits = [0.5, 1, 2, 3, 5, 15, 20, 28, 30, 35];
if ep <= length(fixed_inits)
    init_H = fixed_inits(ep);  % 极端值
else
    init_H = rand() * 35;      % 随机覆盖 0~35
end
```

改进二：合成专家数据增强。我发现模糊控制器在正常工况下表现尚可，但在大误差区域（如初始水位很低需要快速爬升）的数据量不够。为此我手动合成了一批"极端工况"的专家数据——基于比例-微分控制律，确保 NN 学会对大误差做出快速响应：

```
% 合成大误差区域的专家数据 (PD 控制律)
Kp = 2.0; Kd = 0.5;
u_syn = u_eq + Kp * e + Kd * de;  % 前馈 + PD 修正
u_syn = max(0, min(satLevel, u_syn));  % 限幅
```

2.4 训练过程

训练采用标准的 MSE 损失函数，手写 Adam 优化器：

```
L = (1/N) · sum_{i=1}^{N} (y_i - y_hat_i)^2
```

在训练过程中我还做了一个工程上的小处理——**输入输出归一化**。由于各特征的量纲差异很大（误差 `e` 可能达到 ±20，而 `de` 通常在 ±2 以内），不归一化会导致梯度在不同维度上尺度失衡。我对输入减均值除标准差、对输出同样处理，推理时再反归一化：

```
% 输入归一化
x = (features - X_mean) ./ X_std;
% ... NN 前向推理 ...
% 输出反归一化
output = z4 * Y_std + Y_mean;
```

2.5 Derivative Kick 保护

在部署测试中我发现一个问题：当目标水位在 7.5s 从 25 突变到 10 时，`de = e(t) - e(t-1)` 会产生一个巨大的跳变（因为 `sp` 瞬间变了 15），导致 NN 输出剧烈抖动。

我的解决办法是：**检测目标水位是否发生阶跃变化**，若是则将 `de` 强制置零：

```
% 检测目标水位阶跃变化：重置 de 避免 Derivative Kick
if self.last_sp > 0 && abs(sp - self.last_sp) > 1
    de = 0;  % 目标突变时不计算微分
end
```

这个保护机制确保了 NN 在目标切换瞬间不会"被吓到"，而是从零微分状态开始平稳过渡。

2.6 模仿学习的瓶颈

然而，尽管模仿学习的结果看起来还不错，但它的能力上限受制于老师的水平。且其本身对老师的跟踪精度并不完美。即使不断增加训练数据、调整网络结构，最终得分也无法超过模糊控制器本身。且效果无法和老师相当。因此后面我就用强化学习的方法来做，效果相比这个更好。

三、阶段二：强化学习自主优化

3.1 优化目标

该方法的**最终优化目标为**：

```
max_theta E[Score(tau) | pi_theta]
```

这意味着 NN 不再受老师水平的约束，它可以自由探索出更优的控制策略。

3.2 算法选择：可微仿真（Differentiable Simulation）

最终采用的算法是**可微仿真**——将水箱动力学方程直接嵌入神经网络的计算图中，实现端到端的梯度反向传播。

水箱系统的动力学方程是完全可微的：

```
H_{t+1} = H_t + dt · (ip · u_t - op · sqrt(H_t))
```

梯度可以精确计算，直接求梯度就可以。

核心思路：将 1500 步的仿真展开成一张巨大的计算图，NN 的输出 `u_t` 通过动力学影响 `H_{t+1}`，进而影响 `e_{t+1}`，再影响下一步的 NN 输入。整条链路可微，用 Adjoint Method 反传精确梯度。

3.3 前向仿真

训练时不再调用 `Env` 类（其中的 ode45 不可微），而是自己用 **RK4（4阶 Runge-Kutta）积分器**模拟水箱动力学，精度与 ode45 接近且完全可微：

```
% RK4 积分（匹配 ode45 精度）
f_dyn = @(hh) dyn_rhs(hh, u, ip, op);
k1 = dt * f_dyn(Hc);
k2 = dt * f_dyn(Hc + k1/2);
k3 = dt * f_dyn(Hc + k2/2);
k4 = dt * f_dyn(Hc + k3);
H_next = Hc + (k1 + 2*k2 + 2*k3 + k4) / 6;
```

每一步：NN 读取当前状态 → 输出控制量 → RK4 推进水位 → 进入下一步。整条 1500 步轨迹无噪声、确定性地展开。

3.4 损失函数设计（奖励塑形）

直接用环境评分的扣分项作为损失不够——它只惩罚误差和控制偏差，缺乏对"何时到达目标""是否保持稳态"的引导。我设计了一套分层的奖励塑形机制：

（1）基础跟踪损失——和环境评分对齐

```
L_base(t) = (e_t^2 + (u_eq - u_t)^2) · dt
```

（2）到达奖励——越快到达目标水位，奖励越大

当 `|e_t| < 0.5` 时视为首次到达目标，给予一次性奖励：

```
R_reach = 50 × (T_deadline - t_reach) / T_deadline
```

（3）稳态保持惩罚——到达目标后严格约束

一旦到达目标水位，任何偏离（无论超过还是低于）和抖动都受到惩罚：

```
L_steady(t) = 30 · e_t^2 · dt + 20 · (Delta e_t)^2 · dt
```

其中 `30` 和 `20` 是我反复调参后确定的权重——足够大以强制 NN 维持水位，但不至于让梯度爆炸。

（4）超时惩罚——未按时到达的重罚

如果在 Phase 1 结束（7.5s）时未到达 `sp=25`，或 Phase 2 结束（15s）时未到达 `sp=10`，施加 −500 的终端惩罚。

（5）环境分数对齐——每步真实扣分纳入损失

```
L_align(t) = 0.5 × (e_t^2 + (u_eq - u_t)^2) · dt
```

这一项直接将环境评分函数的扣分逻辑复制进训练损失，确保 RL 优化的方向与最终评分一致。

（6）总扣分约束——终端惩罚过度扣分

如果整场仿真的总扣分超过 230 分（即最终得分低于 9770），额外施加惩罚：

```
L_penalty = 0.1 × (总扣分 - 230)
```

3.5 反向传播：Adjoint Method

这是整个实现中最复杂的部分。需要计算 `partial L_total / partial theta`，其中 `theta` 是 NN 的全部 2945 个参数。

由于 `H_{t+1}` 依赖 `u_t`（通过动力学），`u_t` 又依赖 `H_t`（通过 NN 的输入特征 `e_t = sp - H_t`），整条轨迹形成了一个长度为 1500 的递归依赖链。

因此我这里用 **Adjoint Method** 从最后一步反向传播：

定义伴随变量 `lambda_t = partial L / partial H_t`，反向递推：

```
lambda_t = partial L_t / partial H_t
         + lambda_{t+1} · (partial H_{t+1}/partial H_t
         + partial H_{t+1}/partial u_t · partial u_t/partial H_t)
```

其中动力学 Jacobian 为：

```
partial H_{t+1}/partial H_t = 1 - (dt · op) / (2 · sqrt(H_t))
partial H_{t+1}/partial u_t = dt · ip
```

`partial u_t / partial H_t` 则需要单独计算 **NN 的 Jacobian**（注意不能和参数梯度反传混用）：

```
% 独立 NN Jacobian: ∂u_raw/∂x（不含 loss signal）
jac_z4 = Ys;
jac_a3 = jac_z4 * W4';
jac_z3 = jac_a3 .* ((z3>0) + alpha*(z3<=0));
% ... 逐层反传 ...
jac_x  = jac_z1 * W1';  % 1×4: ∂u_raw/∂x

% ∂u_raw/∂H = -(jac_x(1)/Xs(1) + jac_x(2)/Xs(2))
```

每一步 t 的参数梯度信号为：

```
signal_t = partial L_t / partial u_t + lambda_{t+1} · partial H_{t+1}/partial u_t
```

然后用这个标量信号对 NN 做标准反向传播，累积到 `partial L / partial W_i` 和 `partial L / partial b_i` 上。

3.6 训练与验证机制

配置项	值
优化器	Adam (`beta_1=0.9, beta_2=0.999`)
学习率	`5 × 10^-4`
梯度裁剪	max norm = 5.0
Batch size	8（每 epoch 8 条不同初始水位的轨迹）
Epoch 数	500
Warm start	加载模仿学习预训练权重

关键设计：真实环境验证。由于训练用 RK4 仿真、测试用 ode45，两者存在微小数值差异。我每隔 10 个 epoch 用真实的 `Env` 类（ode45）做一次验证，**用验证得分而非训练得分来选择最优模型**，确保保存的权重在真实环境中确实表现最好。

四、实验结果与分析

4.1 模仿学习效果

模仿学习训练完成后，我将 NN 部署到真实环境中进行测试。以下是两个阶段的终端运行截图：

Phase 1 结束（t=7.5s）时的状态：

```
预设水位:25, 当前水位:25.0146
Current Running Time: 7.5 s, Current Score: 9852.6035
```

可以看到，在 7.5 秒时水位已基本到达目标值 25（当前水位 25.0146），得分为 9852.6。

Phase 2 结束（t=15s）时的最终状态：

```
预设水位:10, 当前水位:9.9873
Current Running Time: 15.01 s, Current Score: 9741.2897
```

仿真结束时水位为 9.9873（目标 10），最终得分为 **9741.3。**可以看到 NN 确实学到了模糊控制器的行为模式，两个阶段均能跟踪到目标水位附近。

模糊控制器的控制效果(老师)：

phase1

```
Current Running Time: 7.5 s, Current Score: 9857.757
```

phase2

```
Current Running Time: 9.4 s, Current Score: 9753.2882
Current Running Time: 9.41 s, Current Score: 9753.2882
Current Running Time: 9.42 s, Current Score: 9753.2882
Current Running Time: 9.43 s, Current Score: 9753.2882
Current Running Time: 9.44 s, Current Score: 9753.2882
Current Running Time: 9.45 s, Current Score: 9753.2882
Current Running Time: 9.46 s, Current Score: 9753.2882
Current Running Time: 9.47 s, Current Score: 9753.2882
Current Running Time: 9.48 s, Current Score: 9753.2882
Current Running Time: 9.49 s, Current Score: 9753.2882
Current Running Time: 9.5 s, Current Score: 9753.2882
Current Running Time: 9.51 s, Current Score: 9753.2882
Current Running Time: 9.52 s, Current Score: 9753.2882
```

可以看到NN模仿学习的效果和老师的效果相比还有一定差距。

4.2 强化学习优化后效果

Phase 1 结束（t=7.5s）时的状态:

```
预设水位:25, 当前水位:24.9355
Current Running Time: 7.5 s, Current Score: 9863.9433
```

可以看到，在 7.5 秒时水位已基本到达目标值 25（当前水位 24.9355），得分为 9863.9433。

Phase 2 结束（t=15s）时的最终状态:

```
预设水位:10, 当前水位:10.0008
Current Running Time: 15.01 s, Current Score: 9754.3871
```

仿真结束时水位为 10.0008（目标 10），最终得分为 **9754.3871**。不仅超过了NN模仿学习的结果，也超过了第一次模糊控制器的结果。

五、总结

本次作业我从"模仿学习"出发，逐步过渡到"强化学习自主优化"，构建了一个完整的神经网络水箱液位控制器。我从中学会了面对问题解决问题的能力，也锻炼了我用课程中学习到的深度学习和强化学习的知识去实际解决问题的能力。